

Spesifikasi Tugas Besar

IF1210 Algoritma & Pemrograman 1

2026

Tim Asisten Lab Pemrograman 2023

Versi	: 1
Tgl. Revisi Terakhir	: 6 Mei 2026
Deadline	: 11 Mei 2026, 12.10 WIB (Milestone 1) 25 Mei 2026, 12.10 WIB (Milestone 2)

Daftar Revisi

1. [6 Mei 2026 – 17:00 WIB] [Penambahan Spesifikasi Ketentuan Penanganan Input](#)

Daftar Isi

Daftar Revisi	2
Daftar Isi	3
Overview	5
Latar Belakang	6
Spesifikasi Program	14
Ketentuan Penanganan Input	15
E01 – Format URL	15
E02 – Numerik	15
Spesifikasi Program Utama	16
F00 – Rencana Implementasi	16
F01 – Discover	17
F02 – Search	17
F03 – Open Page	18
F04 – Caching	19
F05 – Page Management (Add, Edit, Delete)	20
add_page <url>	20
edit_page <url>	20
delete_page <url>	21
F06 – Tabs	21
F07 – Back / Forward	24
F08 – Tabs History	26
F09 – Web Graph	27
F10 – Download Manager	28
F11 – Exit	29
Spesifikasi Program Tambahan (STI)	30
S01 – Web Graph	30
S02 – Download Manager	30
S03 – Exit	30
Spesifikasi Program Tambahan (IF & EL & EB)	31
D01 – Web Graph	31
D02 – Download Manager	31

D03 – Load	31
D04 – Save	33
D05 – Exit	34
Struktur Data File	35
Spesifikasi BONUS	39
B01 – Git Best Practice	39
B02 – Global History	42
B03 – Advanced Search	45
B04 – Bookmark Manager	46
BXX – Kreativitas	47
Ketentuan dan Batasan	48
A. Ketentuan Umum	48
B. Batasan Pengerjaan	48
C. Timeline Pengerjaan	50
D. Asistensi	50
E. Demo	51
Pengumpulan dan Deliverables	52
Panduan Instalasi WSL + Makefile	55
A. Windows 10/11	55
B. Mac	56
Referensi	57
Extras	58

Overview

Tugas Besar ini dirancang untuk mengintegrasikan konsep struktur data fundamental ke dalam skenario dunia nyata melalui pengembangan simulasi web browser berbasis terminal.

Melalui tugas ini, diharapkan tidak hanya mampu menerapkan konsep struktur data, tetapi juga mengembangkan kemampuan dalam menyusun kode yang modular, terstruktur, dan mudah dipelihara. Selain itu, pendekatan ini juga mendukung efektivitas kerja tim dalam mengelola dan mengintegrasikan berbagai komponen yang dikembangkan secara kolaboratif.

Latar Belakang

DISCLAIMER: Cerita berikut **TIDAK BERHUBUNGAN** dengan spesifikasi Tugas Besar. Anda dapat menyelesaikan tugas besar tanpa melihat latar belakang sama sekali.

Di Nangor Falls, malam tidak pernah benar-benar sunyi. Selalu ada sesuatu yang bergerak di balik batu, sesuatu yang tidak ingin ditemukan, tetapi juga tidak bisa sepenuhnya bersembunyi. Sudah beberapa minggu terakhir, kota kecil ini dipenuhi kejadian yang sulit dijelaskan. Data dari komputer lokal tiba-tiba menghilang tanpa jejak. Beberapa warga melaporkan kehilangan ingatan jangka pendek. Dan yang paling aneh... tidak ada satupun bukti yang bisa mengarah pada pelaku.

Di dalam *Nangor Shack*, suasana terasa lebih tegang dari biasanya.

Deeper Pinus: "Ini gak masuk akal... semua kejadian ini kayak... terhubung. Tapi gak ada pola yang jelas."

Mebel Pinus: "Atau mungkin ini cuma kota ini yang makin aneh? Maksudku, kita pernah lawan gnome, zombie, bahkan waktu itu—"

Deeper: "Ini beda, Mebel. Ini bukan sekadar 'aneh'. Ini... disengaja."

Deeper meletakkan sebuah catatan di meja. Sebuah peta dengan beberapa titik yang dilingkari merah.

Deeper: "Semua kejadian ini terjadi di sekitar area ini. Dan di tengahnya... ada satu lokasi yang gak pernah ada laporan kejadian apapun."

Stun Pinus: "Tempat yang gak pernah kejadian biasanya justru pusatnya."

Malam itu, mereka memutuskan untuk menyelidiki lokasi tersebut. Sebuah fasilitas tersembunyi di pinggir hutan, hampir tidak terlihat jika tidak tahu harus mencari apa. Lampu redup. Tidak ada papan nama. Terlalu sunyi.

Stun: "Oke, ini jelas bukan tempat piknik keluarga."

Deeper: “Kita harus masuk. Tapi penjagaannya ketat.”

Ia menatap Mebel.

Deeper: “Aku bakal bikin distraksi. Kamu masuk, cari data apapun yang bisa jadi bukti.”

Mebel: “Dan keluar tanpa ketahuan. Ya, klasik.”

Rencana dimulai.

Deeper menciptakan kekacauan di luar—suara keras, cahaya, cukup untuk menarik perhatian para penjaga. Sementara itu, Mebel menyelinap masuk melalui sisi belakang fasilitas.

Di dalam, suasana terasa... tidak alami.

Komputer tua berjajar, layar-layar yang menampilkan data yang terus berubah. Seolah-olah sistem di dalamnya tidak stabil.

Mebel (berbisik pada diri sendiri): “Oke... cepat, cepat...”

Ia menemukan sebuah terminal dengan satu port USB.

Tanpa berpikir panjang, ia menancapkan flashdisk.

Layar berkedip.

Transfer dimulai.

1 file detected... copying...

Mebel: “Serius? Cuma satu file?!”

Sebelum ia sempat memprotes lebih jauh—

[ALARM] INTRUDER DETECTED!!

[di luar tempat Deeper melakukan distraksi]

Deeper: "Uh oh... itu bukan bagian dari rencana."

Ia berlari, mencoba menghindari penjaga yang mulai mengepung area.

Mebel keluar dari dalam, berlari menuju hutan.

Mereka hampir berhasil...

sampai—

SHRKK!

Deeper tersentak.

Sebuah goresan tipis muncul di lengannya, terkena senjata aneh milik salah satu penjaga.

Mebel: "Deeper!"

Deeper: "Aku gak apa-apa—lari!"

Mereka berhasil kembali ke *Nangor Shack*. Untuk sesaat... semuanya terasa aman. Namun itu tidak berlangsung lama.

[Beberapa jam kemudian—]

Mebel: "Deeper, kamu yakin kamu oke?"

Deeper: "...Mebel?" (Ia menatap sekeliling.)

Deeper: "Kita... lagi di mana?"

Seiring waktu, kondisi Deeper semakin memburuk. Ia lupa jalan pulang, lupa siapa Mebel, lupa dirinya sendiri. Memorinya seolah terkunci satu per satu.

Stun: “Aku pernah lihat ini sebelumnya...”

(Mebel menoleh cepat.)

Mebel: “Apa maksudmu?”

Stun: “Ini bukan senjata biasa. Ini... teknologi dari Cipher.”

Stun: “Dia gak nyerang tubuh. Dia nyerang pikiran. Pelan-pelan... sampai korban kehilangan dirinya sendiri.”

Mebel: “...jadi Deeper...”

Stun: “Kalau kita gak ngelakuin sesuatu—dia bakal hilang. Bukan mati... tapi kosong.”

Harapan terakhir mereka ada pada flashdisk itu.

Mebel segera membukanya.

Isinya—

[/secret_url.txt](#)

Satu file yang hanya berisi sebuah link.

Mebel: “Ini apaan sih?! Kita udah hampir mati buat ini, dan cuma dapet link gak guna?! Gabisa dibuka dimana-mana loh!”

Stun hanya diam memperhatikan layar lebih dekat.

Waktu hampir habis. Di sudut ruangan, Deeper hanya duduk diam, menatap kosong. Tidak ada lagi reaksi. Tidak ada lagi pertanyaan. Hanya... hening.

Mebel: “Stun... kita kehabisan waktu...”

Flashdisk itu masih tertancap di komputer. File misterius itu tetap terkunci. Browser biasa tidak bisa membukanya.

Stun: "...mungkin... masih ada satu cara."

Mebel langsung menoleh.

Mebel: "Cara apa?"

Stun: "Kita minta bantuan dia."

Ruangan mendadak terasa lebih dingin.

Mebel langsung tahu siapa yang dimaksud.

Mebel: "NOPE. Gak mungkin. Kita gak bakal minta bantuan ke dia!"

Stun: "Ini bukan soal pilihan lagi." (Ia menatap ke arah Deeper.)

Stun: "Ini soal waktu."

Dengan berat hati... mereka mencoba sesuatu yang seharusnya tidak pernah dilakukan.

Simbol itu digambar.

Ritual kecil dilakukan.

Dan dalam sekejap—

cahaya kuning menyala.

Cipher: "Well, well, well... lama gak ketemu, Pinus."

(Ia melayang santai di udara, seolah semua ini hanya hiburan baginya.)

Mebel: "Kita gak punya waktu buat main-main. Kita butuh bantuan."

Cipher: "Bantuan? Dari aku? Wow... ini baru menarik." (menyeringai)

(Ia melirik ke arah Deeper.)

Cipher: "Oh... jadi kalian akhirnya ketemu sama mainan lama aku."

Stun: "Kalau kamu tahu cara menghentikan ini—bilang sekarang."

Cipher tidak langsung menjawab.

Ia melihat layar komputer.

File misterius itu.

Lalu—

ia tertawa kecil.

Cipher: "Kalian bahkan belum bisa buka ini? Pathetic."

Mebel: "Kalau kamu cuma mau ngejek—keluar aja!"

Untuk sesaat, Cipher terdiam. Lalu... sesuatu yang aneh terjadi.

Ia tidak mengejek lagi.

Cipher: "...oke. Sekarang mood-ku lagi bagus."

Cahaya di sekitarnya berubah.

Ia mengangkat tangannya—dan seketika, layar komputer berkedip.

Kode mulai muncul.

Baris demi baris.

Cipher: "Kalian butuh browser khusus. Bukan browser biasa." (menyeringai)

Cipher: "Dan jelas... kalian gak bakal sempet bikin dari nol."

Stun: "Jadi?"

Cipher: "Anggap aja ini... bantuan kecil."

Di layar, terbentuk sebuah struktur awal.

Sebuah **kerangka program**.

Belum lengkap.

Belum bisa digunakan sepenuhnya.

Tapi cukup untuk memulai.

Cipher: "Aku kasih kalian fondasinya."

(Ia menunjuk ke layar)

Cipher: "Tapi sisanya... kalian yang selesaikan."

Mebel: "Kenapa kamu bantu kita?"

Cipher: "Siapa bilang aku bantu?"

Cipher: "Aku cuma pengen lihat... kalian bisa mendapat yang terbaik di **Tugas Besar IF1210 Algoritma & Pemrograman-1 2026**."

Dalam sekejap—Cipher menghilang.

Meninggalkan satu hal, sebuah **spesifikasi** untuk browser misterius tersebut.

stun: "...aku gak suka ini."

Mebel: "Tapi ini satu-satunya yang kita punya."

Sekarang—

semua bergantung pada kode itu.

Sebuah kerangka awal yang harus dilanjutkan.

Sebuah sistem yang harus diselesaikan.

Dan waktu... terus berjalan.

Bangunlah browsernya. Selesaikanlah sistemnya. Bukalah file itu.

Karena mungkin—

di dalamnya... ada cara untuk mengembalikan Deeper.

Spesifikasi Program

Terdapat kebutuhan fungsional wajib yang diperlukan, seperti yang tertera di bawah. Tampilan atau interface dari sistem dibebaskan, silahkan berkreasi; output tidak harus persis seperti contoh, yang penting spesifikasi terpenuhi. Kreativitas interface akan menjadi pertimbangan penilaian. Perlu diperhatikan bahwa yang menjadi penekanan bukan hanya pada **alur jalannya program**, tetapi juga pada **validasi aksi** yang dilakukan.

Banyaknya fungsionalitas yang wajib diimplementasikan bukan berarti program yang akan dibuat juga akan panjang dan kompleks. Dengan pembuatan program yang **modular** dengan **fungsi/prosedur yang jelas**, program dapat dibuat secara relatif lebih **singkat dan sederhana**. Silakan pahami spesifikasi Tugas Besar ini dengan baik sebelum melakukan pengerjaan. Ingat, ini merupakan Tugas Besar berkelompok, sehingga aturlah sedemikian mungkin agar tugas ini tidak dibebankan kepada beberapa orang saja.

Notes:

Untuk mempermudah pengerjaan tugas besar, kami telah memberikan contoh pada masing-masing spesifikasi fungsional. Namun, keluaran dan masukan program dapat disesuaikan dengan kreativitas masing-masing. Adapun beberapa informasi tambahan terkait contoh yang kami berikan adalah sebagai berikut:

- Teks Normal berarti output
- **Teks Bold** berarti input

WARNING: CICIL Pengerjaan dari jauh-jauh hari dan kerjakan dengan konsisten. Jika mengerjakan dekat dengan deadline, sangat beresiko tugas besar ini tidak selesai dengan baik!

Ketentuan Penanganan Input

Untuk mengurangi ambiguitas pada masukan pengguna, setiap input wajib mengikuti batasan yang telah ditentukan. Apabila input pengguna berada di luar ketentuan, program harus menanganinya dengan memberikan output bahwa masukan tidak valid.

Berikut ketentuan validasi input yang harus diperhatikan:

E01 – Format URL

Spesifikasi terdampak: **F02, F03, F05, F09, F10**

Pastikan input URL divalidasi sesuai aturan [regex](#) berikut:

```
^(?!-)[A-Za-z0-9-]{1,63}(?!-)(\.[A-Za-z]{2,})+$
```

Silakan melakukan eksplorasi mandiri untuk memahami makna dari regex tersebut. Namun, validasi tidak boleh dilakukan menggunakan library regex. Implementasikan proses validasi tersebut secara manual di dalam program kalian.

E02 – Numerik

Spesifikasi terdampak: **F06, F08, F09**

Pastikan setiap nilai X yang diterima program merupakan bilangan bulat positif dengan panjang maksimal 6 digit, atau memiliki nilai maksimum 999999. Masukan yang tidak memenuhi ketentuan tersebut harus dianggap sebagai input tidak valid.

Spesifikasi Program Utama

F00 – Rencana Implementasi

Silakan buat sebuah rencana implementasi ADT dan rencana implementasi materi dalam dokumen. Rencana implementasi ini nantinya akan kalian jadikan panduan pengerjaan program. Berikut beberapa materi atau ADT yang harus kalian implementasikan:

- ADT Sederhana
- ADT List
- ADT Linked List
- ADT Matrix (Khusus IF & EL & EB)
- ADT Set
- ADT Map
- ADT Stack
- ADT Queue
- ADT Graph (Khusus STI)
- File External (Khusus IF & EL & EB)
- Fungsi & Prosedur
- Array Search, Sort, Filter (Khusus STI → wajib ada binary search)

Notes:

Kalian diperbolehkan menggabungkan dua atau lebih ADT & materi dalam 1 implementasi

F01 – Discover

Untuk membantu eksplorasi, pengguna dapat melihat beberapa URL dengan perintah **discover**. Perintah ini akan menampilkan 5 URL secara acak dari database halaman web yang telah dimuat sebelumnya. Pemilihan URL acak wajib menggunakan algoritma [Linear Congruential Generator \(LCG\)](#) yang diimplementasikan sendiri tanpa bantuan library eksternal (seperti `rand()` atau `random()`). Jika jumlah total halaman web kurang dari 5, maka tampilkan seluruh halaman yang tersedia.

```
# DISCLAIMER: Tampilan/interface dibebaskan, berikut hanya contoh saja

>>> discover

Berikut adalah beberapa halaman yang mungkin menarik untukmu:
- mysteryshack.com
- gideon-tent.org
- journal-3.nf
- batu-ajaib-asli.com
- topi-pinus-original.com
```

F02 – Search

Agar pengguna dapat menemukan web yang ingin dituju, program perlu mengimplementasikan fungsionalitas search.

Pengguna dapat mencari URL web yang telah di-load dengan perintah **'search <query>'**. Pencarian dilakukan dengan mencocokkan string query pada string URL, dimulai dari awal string URL. Perintah hanya perlu menerima 1 query. Jika terdapat lebih dari 1 query di satu perintah, alur program dibebaskan.

```
# DISCLAIMER: Tampilan/interface dibebaskan, berikut hanya contoh saja

# KASUS 1: User mencari halaman web dengan query "mystery".
# Dua Web ditemukan:
# - mysteryshack.com ditemukan karena URL dimulai dari string "mystery"
# - mystery-journals.net ditemukan karena URL dimulai dari string "mystery"

>>> search mystery

Search result(s) for "mystery":
- mysteryshack.com
- mystery-journals.net
```

```
# KASUS 2: User mencari halaman web dengan query "secret-society".  
# Web tidak ditemukan.
```

```
>>> search secret-society
```

```
Search result(s) for "secret-society":  
Tidak Ditemukan
```

(NB: Lihat juga bonus [Advanced Search](#).)

F03 – Open Page

Pengguna dapat membuka halaman web dengan perintah **open <url>**. Sistem akan mencari isi konten dari URL tersebut. Jika URL sudah di-[cache](#), ambil konten dari *cache*. Jika belum di-*cache*, lakukan linear search pada kumpulan halaman web (Gunakan **ADT Set** untuk menyimpan daftar halaman web) untuk mendapatkan konten URL tersebut. Saat membuka halaman web, pengguna tidak dapat mengakses *command* pada menu utama. *Command* yang bisa digunakan saat membuka halaman adalah: **home** untuk kembali ke menu utama, [back](#) atau [forward](#), atau **openlinked <x>** untuk membuka [linked pages](#).

```
# DISCLAIMER: Tampilan/interface dibebaskan, berikut hanya contoh saja
```

```
# KASUS 1: User membuka url halaman web yang terdaftar. Halaman web memiliki linked pages.  
Url belum di-cache.
```

```
>>> open mysteryshack.com
```

```
[Status: Cache-Miss] Mengambil data dari database...
```

```
SELAMAT DATANG DI MYSTERY SHACK!
```

```
Tempat paling ajaib di Nangor Falls (dan satu-satunya yang punya diskon 0%!).
```

```
Lihatlah Jackalope asli! Lihatlah batu yang menyerupai wajah Stun!
```

```
Peringatan: Barang yang sudah dibeli tidak dapat dikembalikan, apalagi kalau Anda baru  
sadar itu cuma barang rongsokan yang ditempel lem.
```

```
Linked pages:
```

```
[1] batu-ajaib-asli.com
```

```
[2] topi-pinus-original.com
```

```
# KASUS 2: User membuka url halaman web yang terdaftar. Halaman web TIDAK memiliki linked  
pages. Url sudah di-cache.
```

```
>>> open topi-pinus-original.com
```

```
[Status: Cache-Hit] Mengambil data dari cache...
```

TOPI PINUS IKONIK - SEPERTI YANG DIPAKAI ANAK ITU!
Stok terbatas! Dapatkan topi biru-putih dengan logo pohon pinus yang melegenda.
Dijamin 100% serat kain asli (mungkin).

KASUS 3: User membuka url halaman web yang tidak terdaftar.

>>> open bill.cipher.nf

404 Not Found! Halaman tidak ditemukan.

F04 – Caching

Untuk meningkatkan efisiensi waktu akses, program harus mengimplementasikan sistem *caching* saat membuka halaman web. *Cache* akan menyimpan pasangan *key-value* antara URL halaman web dan kontennya (Gunakan **ADT Map** untuk implementasi *cache*). *Cache* diperlukan agar waktu akses terhadap URL yang sering atau baru saja diakses menjadi jauh lebih cepat. Dengan menyimpan data secara sementara di dalam *cache*, program dapat meminimalisir proses pencarian berulang ke dalam keseluruhan data halaman web.

Namun, kapasitas *cache* tidak dapat dibiarkan tak terbatas. Jika seluruh halaman yang pernah dibuka terus disimpan, *cache* hanya akan menjadi duplikat dari daftar halaman web yang ada dan kehilangan efisiensinya sebagai struktur data akses cepat. Oleh karena itu, program harus membatasi kapasitas *cache*. Gunakan suatu nilai variabel statis yang telah ditentukan sejak awal (misal `CACHE_MAX_AMOUNT = 10`) untuk kapasitas *cache*. Lebih jelasnya, aturan kerja *cache* adalah sebagai berikut:

1. Pengguna menjalankan perintah **open <url>**.
2. Jika URL yang dicari pengguna ditemukan di dalam *cache* (disebut *Cache-Hit*), konten langsung dimuat dan ditampilkan dari *cache*.
3. Jika URL tidak ditemukan di *cache* (disebut *Cache-Miss*), program harus mencari data dari daftar halaman web utama, menampilkan kontennya, kemudian **menyimpannya ke dalam cache**.
4. Karena terdapat kapasitas maksimal, maka jika *cache* sudah terisi penuh dan terdapat halaman baru yang perlu disimpan, program wajib **membuang salah satu item lama** untuk memberikan ruang bagi data baru. [Algoritma](#)

[pemilihan](#) item yang dibuang (misal: *First-In First-Out*, *Least Recently Used*, atau lainnya) dibebaskan sesuai dengan kreativitas masing-masing.

F05 – Page Management (Add, Edit, Delete)

Pengguna dapat mengelola basis data halaman web secara dinamis melalui perintah ***add_page <url>***, ***edit_page <url>***, dan ***delete_page <url>***. Id dari halaman baru dan juga id dari hubungan antar halaman (*linked pages*) menggunakan aturan ***auto-increment*** ($ID_{baru} = ID_{terbesar} + 1$). Program wajib menjaga konsistensi data antara database utama, Cache, dan Web Graph agar tidak terjadi *stale data* (data basi) atau *dead links* (tautan ke halaman yang sudah tidak ada).

add_page <url>

```
# DISCLAIMER: Tampilan/interface dibebaskan, berikut hanya contoh saja

# KASUS 1: Menambah halaman yang sudah terdaftar.
>>> add_page mysteryshack.com
Sudah terdapat halaman dengan url mysteryshack.com. Gunakan url lain yang belum terdaftar!

# KASUS 2: Menambah halaman yang belum terdaftar.
>>> add_page mysteryshock.com
Masukkan konten (Akhirinya dengan titik '.' di baris baru):
>>> SELAMAT DATANG DI MYSTERY SHOCK!
>>> Tempat paling ajaib di Nangor Falls (dan satu-satunya yang punya diskon 0%!).
>>> Lihatlah Jackalope asli! Lihatlah batu yang menyerupai wajah Stun!
>>> Peringatan: Barang yang sudah dibeli tidak dapat dikembalikan, apalagi kalau Anda baru
sadar itu cuma barang rongsokan yang ditempel lem.
>>> Tenang, ini bukan link web phishing kok!
>>> .
Masukkan linked pages (Ketik 'DONE' jika sudah selesai):
>>> topi-pinus-kw.com
>>> salah-url.id
URL tidak ditemukan!
>>> DONE
Halaman mysteryshock.com berhasil ditambahkan!
```

edit_page <url>

```
# DISCLAIMER: Tampilan/interface dibebaskan, berikut hanya contoh saja

# KASUS 1: Mengedit halaman yang belum terdaftar.
>>> edit_page mysterysheck.com
Tidak ada halaman dengan url mysterysheck.com!

# KASUS 2: Menambah halaman yang belum terdaftar.
```

```

>>> edit_page mysteryshock.com
[Status: Cache-Hit] Mengambil data dari cache...

Konten saat ini:
SELAMAT DATANG DI MYSTERY SHOCK!
Tempat paling ajaib di Nangor Falls (dan satu-satunya yang punya diskon 0%!).
Lihatlah Jackalope asli! Lihatlah batu yang menyerupai wajah Stun!
Peringatan: Barang yang sudah dibeli tidak dapat dikembalikan, apalagi kalau Anda baru
sadar itu cuma barang rongsokan yang ditempel lem.
Tenang, ini bukan link web phising kok!

Linked pages:
[1] topi-pinus-kw.com

Masukkan konten baru (akhiri dengan '.' atau ketik '.' saja jika tidak ingin mengubah
konten):
>>> .
Masukkan linked pages baru (Ketik 'DONE' jika sudah selesai, atau ketik 'SKIP' jika tidak
ingin mengubah linked pages):
>>> SKIP
Halaman mysteryshock.com berhasil diperbarui!

```

delete_page <url>

```

# DISCLAIMER: Tampilan/interface dibebaskan, berikut hanya contoh saja

# KASUS 1: Menghapus halaman yang belum terdaftar.
>>> delete_page mysterysheck.com
Tidak ada halaman dengan url mysterysheck.com!

# KASUS 2: Menambah halaman yang belum terdaftar.
>>> delete_page mysteryshock.com
[Status: Cache-Hit] URL ditemukan di cache dan telah dibersihkan.
Membersihkan relasi linked pages...

Halaman mysteryshock.com berhasil dihapus!

>>> open mysteryshock.com
404 Not Found! Halaman tidak ditemukan.

```

F06 – Tabs

Program ini mengimplementasikan multiple tabs. Pengguna dapat membuka tab baru dengan **newtab**, menutup tab yang aktif dengan **closetab**, mengecek daftar tab yang ada dengan **checktab**, serta berpindah antar tab dengan **prevtab** atau **nexttab**. Pada setiap tab, pengguna dapat menjalankan perintah seperti search, open page, navigasi back/forward, download, serta open link.

Saat program pertama kali dijalankan, satu tab akan terinisialisasi secara *default*, dan *current* tab akan selalu dimulai dari tab pada posisi pertama. Implementasi nama tab dibuat dalam format 'TAB<counter>' di mana counter akan *auto-increment* setiap ada tab baru sehingga tidak perlu memasukkan input nama tab. Gunakan suatu nilai variabel statis yang telah ditentukan sejak awal (misal `TABS_MAX_AMOUNT = 10`) untuk kapasitas tab.

```
# DISCLAIMER: Tampilan/interface dibebaskan, berikut hanya contoh saja
```

```
# KASUS 1: User menghapus tab saat hanya ada 1 tab.
```

```
>>> checktab
```

```
List of tab(s):
```

```
[1] TAB1
```

```
Current tab: TAB1
```

```
>>> closetab
```

```
ERROR: Tidak bisa menutup tab, tab minimal berjumlah 1!
```

```
# KASUS 2: User menambahkan tab baru saat program pertama dijalankan.
```

```
>>> checktab
```

```
List of tab(s):
```

```
[1] TAB1
```

```
Current tab: TAB1
```

```
>>> newtab
```

```
Tab baru (TAB2) berhasil dibuat!
```

```
>>> newtab
```

```
Tab baru (TAB3) berhasil dibuat!
```

```
>>> checktab
```

```
List of tab(s):
```

```
[1] TAB1
```

```
[2] TAB2
```

```
[3] TAB3
```

```
Current tab: TAB1
```

```
# KASUS 3: User ingin berpindah tab.
```

```
Gunakan command:
```

```
prevtab <jarak-dari-current-tab> atau nexttab <jarak-dari-current-tab>
```

```
>>> checktab
```

```
List of tab(s):
```

```
[1] TAB1
```

```
[2] TAB2
```

```
[3] TAB3
```

```
Current tab: TAB1
```

```
>>> nexttab 2
```

```
Tab saat ini berhasil diganti ke TAB3.
```

```
>>> checktab
```

```
List of tab(s):
```

```
[1] TAB1
```

```
[2] TAB2
```

```
[3] TAB3
```

Current tab: TAB3

>>> **prevtab 1**

Tab saat ini berhasil diganti ke TAB2.

>>> **checktab**

List of tab(s):

[1] TAB1

[2] TAB2

[3] TAB3

Current tab: TAB2

KASUS 4: User ingin berpindah ke tab yang tidak valid.

>>> **checktab**

List of tab(s):

[1] TAB1

[2] TAB2

[3] TAB3

Current tab: TAB2

>>> **nexttab 2**

ERROR: Posisi tab tidak valid!

KASUS 5: User ingin menghapus tab dan menambahkan tab baru.

>>> **checktab**

List of tab(s):

[1] TAB1

[2] TAB2

[3] TAB3

Current tab: TAB2

>>> **closetab**

TAB2 berhasil ditutup.

>>> **checktab**

List of tab(s):

[1] TAB1

[2] TAB3

Current tab: TAB3

>>> **newtab**

Tab baru (TAB4) berhasil dibuat!

>>> **checktab**

List of tab(s):

[1] TAB1

[2] TAB3

[3] TAB4

Current tab: TAB3

KASUS 6: User ingin menambahkan tab baru, tetapi jumlah tab saat ini sudah mencapai TABS_MAX_AMOUNT.

>>> **newtab**

ERROR: Jumlah tab tidak bisa melebihi batas maksimum!

Perhatikan bahwa tab baru pasti ditambahkan di akhir list dan pengguna hanya bisa menutup tab yang sedang dibuka.

F07 – Back / Forward

Pengguna dapat menavigasi halaman web yang pernah dibuka sebelumnya dengan *Command* **back** atau **forward**. *Command* **back** akan membuka halaman yang sebelumnya telah dikunjungi dan **forward** akan membuka kembali halaman berikutnya setelah melakukan **back**, layaknya *browser* pada umumnya. Jika pengguna membuka halaman baru secara manual (bukan melalui back/forward) saat berada di tengah-tengah history, maka seluruh history 'forward' pada tab tersebut harus dihapus. **Perhatikan bahwa setiap Tab memiliki Back/Forward history-nya masing-masing**. Oleh karena itu, pastikan bahwa ketika pengguna pindah Tab, *history* navigasinya sesuai dengan tab yang aktif dan tidak tercampur dengan tab lainnya.

Berikut merupakan contoh kasus-kasus yang mungkin terjadi untuk **back navigation**:

```
# DISCLAIMER: Tampilan/ interface dibebaskan, berikut hanya contoh saja
```

```
# KASUS 1: User mencoba melakukan back tanpa membuka halaman apapun sebelumnya.
```

```
>>> back
```

```
ERROR: BACK TIDAK BISA DIJALANKAN KARENA TIDAK ADA HALAMAN SEBELUMNYA!
```

```
# KASUS 2: User mencoba melakukan back, tetapi telah di halaman paling awal.
```

```
>>> back
```

```
ERROR: BACK TIDAK BISA DIGUNAKAN LAGI KARENA TIDAK ADA HALAMAN SEBELUMNYA!
```

```
# kembali menampilkan halaman paling awal
```

```
SELAMAT DATANG DI MYSTERY SHACK!
```

```
Tempat paling ajaib di Nangor Falls (dan satu-satunya yang punya diskon 0%!).
```

```
Lihatlah Jackalope asli! Lihatlah batu yang menyerupai wajah Stun!
```

```
Peringatan: Barang yang sudah dibeli tidak dapat dikembalikan, apalagi kalau Anda baru sadar itu cuma barang rongsokan yang ditempel lem.
```

```
Linked pages:
```

```
[1] batu-ajaib-asli.com
```

```
[2] topi-pinus-original.com
```

```
# KASUS 3: User mencoba melakukan back dan telah membuka halaman sebelumnya.
```

```
>>> back
```

```
BACK: KEMBALI KE HALAMAN mysteryshack.com
```

```
SELAMAT DATANG DI MYSTERY SHACK!
```

```
Tempat paling ajaib di Nangor Falls (dan satu-satunya yang punya diskon 0%!).
```


Lihatlah Jackalope asli! Lihatlah batu yang menyerupai wajah Stun!
Peringatan: Barang yang sudah dibeli tidak dapat dikembalikan, apalagi kalau Anda baru sadar itu cuma barang rongsokan yang ditempel lem.

Linked pages:

[1] batu-ajaib-asli.com

[2] topi-pinus-original.com

Berikut merupakan contoh kasus-kasus yang mungkin terjadi untuk **forward navigation**:

```
# DISCLAIMER: Tampilan/ interface dibebaskan, berikut hanya contoh saja
```

```
# KASUS 1: User mencoba melakukan forward tanpa membuka halaman apapun sebelumnya.
```

```
>>> forward
```

```
ERROR: FORWARD TIDAK BISA DIJALANKAN KARENA TIDAK ADA HALAMAN SELANJUTNYA!
```

```
# KASUS 2: User mencoba melakukan forward, tetapi belum melakukan back atau back history terpotong seperti saat membuka halaman baru secara manual.
```

```
>>> forward
```

```
ERROR: FORWARD TIDAK BISA DIJALANKAN KARENA TIDAK ADA HALAMAN SELANJUTNYA!
```

```
# kembali menampilkan halaman paling akhir
```

```
SELAMAT DATANG DI MYSTERY SHACK!
```

```
Tempat paling ajaib di Nangor Falls (dan satu-satunya yang punya diskon 0%).
```

```
Lihatlah Jackalope asli! Lihatlah batu yang menyerupai wajah Stun!
```

```
Peringatan: Barang yang sudah dibeli tidak dapat dikembalikan, apalagi kalau Anda baru sadar itu cuma barang rongsokan yang ditempel lem.
```

```
Linked pages:
```

```
[1] batu-ajaib-asli.com
```

```
[2] topi-pinus-original.com
```

```
# KASUS 3: User mencoba melakukan forward dan ada back history (sudah melakukan back sebelumnya).
```

```
>>> forward
```

```
FORWARD: KEMBALI KE HALAMAN mysteryshack.com
```

```
SELAMAT DATANG DI MYSTERY SHACK!
```

```
Tempat paling ajaib di Nangor Falls (dan satu-satunya yang punya diskon 0%).
```

```
Lihatlah Jackalope asli! Lihatlah batu yang menyerupai wajah Stun!
```

```
Peringatan: Barang yang sudah dibeli tidak dapat dikembalikan, apalagi kalau Anda baru sadar itu cuma barang rongsokan yang ditempel lem.
```

```
Linked pages:
```

```
[1] batu-ajaib-asli.com
```

```
[2] topi-pinus-original.com
```

F08 – Tabs History

Pengguna dapat melihat *history* halaman pada tab aktif. *History* ini menampilkan urutan halaman yang pernah dibuka dalam tab tersebut. Berdasarkan urutan tersebut, Pengguna dapat menggunakan command '**back <X>**' atau '**forward <X>**' dengan **X** adalah jumlah langkah navigasi yang ingin dilakukan. Contoh penggunaannya adalah sebagai berikut:

```
>>> view_tab_history
```

```
[0] cipher.com
[1] gideon.com
[2] mysteryshack.com <- YOU ARE HERE
[3] pines.com
```

SELAMAT DATANG DI MYSTERY SHACK!

Tempat paling ajaib di Nangor Falls (dan satu-satunya yang punya diskon 0%!).

Lihatlah Jackalope asli! Lihatlah batu yang menyerupai wajah Stun!

Peringatan: Barang yang sudah dibeli tidak dapat dikembalikan, apalagi kalau Anda baru sadar itu cuma barang rongsokan yang ditempel lem.

Linked pages:

```
[1] batu-ajaib-asli.com
[2] topi-pinus-original.com
```

```
>>> BACK 2
```

```
BACK: KEMBALI KE HALAMAN cipher.com # INDEX 2 -> 0
```

```

$$$$$$\  $$$$$$\  $$$$$$$\  $$\  $$\  $$$$$$$\  $$$$$$$\
$$  __$$\  \_$$  _|$$  __$$\  $$ |  $$ |$$  _____|$$  __$$\
$$ /  \__|  $$ |  $$ |  $$ |$$ |  $$ |$$ |  _____|$$ |  $$ |
$$ |  _____|  $$$$$$$$  |$$$$$$$$  |$$$$$$\  $$$$$$$  |
$$ |  _____|  $$$  ____/  $$  __$$  |$$  __|  $$  __$$<
$$ |  $$\  $$ |  $$ |  _____|  $$$  |  $$ |$$ |  _____|  $$$  |
\$$$$$$$  |$$$$$$$  $$ |  _____|  $$$  |  $$$  |$$$$$$$$\  $$ |  $$ |
 \_____/  \_____/  \__|  \__|  \__|  \_____/  \__|  \__|
```

```
# SILAHKAN PIKIRKAN KASUS-KASUS YANG MUNGKIN TERJADI DENGAN ADANYA FITUR INI.
```

F09 – Web Graph

Pengguna dapat melihat relasi antar halaman web melalui Web Graph yang memuat data dari file [linked_pages.csv](#). Data yang dimuat akan disimpan dalam struktur data tertentu. Silahkan lihat spesifikasi khusus masing-masing berikut:

[Spesifikasi Khusus IF, EL, EB]

[Spesifikasi Khusus STI]

Saat pengguna membuka sebuah halaman, **program akan memproses data relasi tersebut untuk mencari dan menampilkan semua halaman yang tertaut** (*linked pages*) dari halaman yang sedang diakses. Daftar tautan ini disajikan dengan format penomoran berurutan, seperti '**[1]** <url-1>', '**[2]** <url-2>', dan seterusnya. Selanjutnya, pengguna dapat melakukan navigasi ke halaman tertaut tersebut dengan memasukkan command **openlinked <x>** dengan **x** merujuk pada nomor indeks dari daftar tautan yang sedang ditampilkan di layar.

Berikut merupakan contoh kasus-kasus yang mungkin terjadi untuk **Web Graph**.

```
# DISCLAIMER: Tampilan/ interface dibebaskan, berikut hanya contoh saja

# KASUS 1: User mencoba membuka linked page dengan indeks yang valid.
# Perintah 1: User membuka salah satu halaman web yang memiliki linked page
>>> open mysteryshack.com

SELAMAT DATANG DI MYSTERY SHACK!
Tempat paling ajaib di Nangor Falls (dan satu-satunya yang punya diskon 0%!).
Lihatlah Jackalope asli! Lihatlah batu yang menyerupai wajah Stun!
Peringatan: Barang yang sudah dibeli tidak dapat dikembalikan, apalagi kalau Anda baru
sadar itu cuma barang rongsokan yang ditempel lem.

Linked pages:
[1] batu-ajaib-asli.com
[2] topi-pinus-original.com

# Perintah 2: User membuka salah satu linked page
>>> openlinked 2

TOPI PINUS IKONIK - SEPERTI YANG DIPAKAI ANAK ITU!
Stok terbatas! Dapatkan topi biru-putih dengan logo pohon pinus yang melegenda.
Dijamin 100% serat kain asli (mungkin).

# KASUS 2: User mencoba akses linked page pada halaman web yang tidak memiliki tautan.
>>> openlinked 1

ERROR: HALAMAN TIDAK MEMILIKI TAUTAN YANG BISA DIBUKA!

TOPI PINUS IKONIK - SEPERTI YANG DIPAKAI ANAK ITU!
```

Stok terbatas! Dapatkan topi biru-putih dengan logo pohon pinus yang melegenda.
Dijamin 100% serat kain asli (mungkin).

KASUS 3: User mencoba akses linked page tanpa membuka halaman apapun.

>>> openlinked 1

ERROR: COMMAND HANYA DAPAT DIGUNAKAN SAAT HALAMAN WEB TERBUKA!

F10 – Download Manager

Pengguna dapat mengunduh halaman web ke dalam sistem lokal. Proses unduhan ini tidak terjadi secara instan, melainkan disimulasikan menggunakan sistem waktu berbasis **tick** (detik/langkah). Seluruh permintaan unduhan akan ditampung di dalam sebuah **Queue**, di mana **sistem hanya dapat mengunduh satu halaman pada satu waktu**. Dengan keterbatasan memori pada browser ini, antrean unduhan perlu dibatasi. Gunakan suatu nilai variabel statis yang telah ditentukan sejak awal (misal `DOWNLOAD_MAX_AMOUNT = 5`) untuk kapasitas antrean.

Perintah yang dapat pengguna masukkan meliputi:

1. **download <url>**

Menambahkan halaman ke dalam antrean unduhan, dimana setiap halaman web yang diunduh akan membutuhkan waktu penyelesaian sebanyak **N ticks** ($N = \frac{\text{length}(\text{URL})}{5} + 2$, dimana **hasil pembagian dibulatkan ke bawah**). Jika antrean kosong, unduhan akan langsung masuk ke antrean berstatus aktif dan menunggu tick berjalan. Jika antrean tidak kosong, unduhan baru akan ditambahkan ke bagian belakang antrean.

2. **tick**: Memajukan waktu sistem sebanyak 1 langkah. Saat tick dipanggil, sistem akan melihat download terdepan pada antrean.

[\[Spesifikasi Khusus IF, EL, EB\]](#)

[\[Spesifikasi Khusus STI\]](#)

Berikut merupakan contoh keluaran dari **Download Manager**:

```

# DISCLAIMER: Tampilan/ interface dibebaskan, berikut hanya contoh saja

# Kasus 1

# Perintah 1: Pengguna melakukan download ketika belum ada download aktif
>>> download youtube.com

Download youtube.com (5 ticks)

# Perintah 2: Pengguna melakukan download ketika sudah terdapat antrean download aktif
>>> download github.com

Download github.com (4 ticks) -> antrian no 2, 5 ticks masih tertunda dari antrian sebelumnya

# Perintah 3: Pengguna melakukan tick ketika terdapat antrean
>>> tick

Downloading youtube.com... (4 ticks tersisa)

#... (setelah 4 ticks berikutnya) ...

# Perintah 4: Pengguna melakukan tick dan download selesai
>>> tick
# Output untuk IF, EL, EB
youtube.com selesai didownload, ke file youtube.txt.
Lanjut downloading github.com... (4 ticks tersisa)

# Perintah 4: Output untuk STI
>>> tick

youtube.com selesai terdownload!
Lanjut downloading github.com... (4 ticks tersisa)

# Kasus 2: Pengguna melakukan download ketika antrean penuh
>>> download youtube.com

Download tidak diterima, antrian sudah penuh.

# Kasus 3: Pengguna melakukan tick pada saat antrean download kosong
>>> tick

Antrian download saat ini kosong.

```

F11 – Exit

Pengguna tentu saja harus dapat keluar dari aplikasi web browser. Pengguna dapat keluar dari program dengan menggunakan command **exit** pada halaman utama.

[\[Spesifikasi Khusus IF, EL, EB\]](#)

[\[Spesifikasi Khusus STI\]](#)

Spesifikasi Program Tambahan (STI)

S01 – Web Graph

Penyimpanan data **menggunakan ADT Graph**.

S02 – Download Manager

Ketika unduhan selesai dilakukan, sistem hanya perlu menampilkan output “<URL> **selesai terdownload!**”.

S03 – Exit

Ketika pengguna menggunakan perintah **exit**, cukup tampilkan pesan sebelum keluar dari program.

Spesifikasi Program Tambahan (IF & EL & EB)

D01 – Web Graph

Penyimpanan data **menggunakan ADT Matrix** (*adjacency matrix*). Gunakan suatu nilai variabel statis yang telah ditentukan sejak awal (misal `MAX_WEB_PAGES = 100`) untuk kapasitas *adjacency matrix*. Jika jumlah halaman web hasil load atau ***add_page*** melebihi batas ini, sistem harus memberikan peringatan dan menolak penambahan halaman baru demi menjaga kestabilan memori.

D02 – Download Manager

Ketika unduhan selesai dilakukan, sistem membuat keluaran file dengan format penamaan **<URL>.txt** dengan konten file sesuai dengan konten pada web yang di-download. **Postfix URL pada nama file akan dipotong** (contohnya .com, .edu, .id, dll).

D03 – Load

Aplikasi memerlukan sekumpulan data *web pages*, relasi *linked pages*, dan data konfigurasi untuk dapat berfungsi. Namun, data-data ini tidak baik disimpan langsung di dalam kode program. Oleh karena itu, data-data ini akan disimpan dalam file berbentuk *comma-separated value* (.csv file) dan *text file* (.txt file).

Load dilakukan secara otomatis ketika program dijalankan pertama kali melalui `makefile`.

```
# Contoh isi makefile

CONFIG_FOLDER ?= config/ # Ganti dengan nama folder konfigurasi default kalian
APP = main # Bisa diganti dengan nama aplikasi kalian

# ...

run:
    ./${APP} ${CONFIG_FOLDER}
```

```
# ...

# Contoh menjalankan makefile

# Contoh 1
terminal@terminal ~/tubes $ make run

# Contoh 2
terminal@terminal ~/tubes $ make run CONFIG_FOLDER=custom_dir/

# Syntax ?= dalam makefile digunakan agar bisa assign value ke suatu variabel (Bisa make
run seperti contoh 2)
```

Aplikasi juga dapat melakukan load ketika sedang dijalankan untuk melakukan sinkronisasi data (data yang disimpan mungkin berubah saat aplikasi dijalankan) atau load data dari folder konfigurasi yang lain. Saat berada di menu utama, gunakan perintah **load <folder/>**. Aplikasi akan melakukan load ulang dan memperbarui keadaan aplikasi berdasarkan data yang di-load.

Note: Silahkan sesuaikan Makefile yang ada di repository.

```
# DISCLAIMER: Tampilan/interface dibebaskan, berikut hanya contoh saja

# KASUS: User melakukan load ulang

>>> load config/

Loading new data from config/ folder...
New data loaded

# KASUS: Folder tidak ditemukan

>>> load config/

Loading new data from config/ folder...
Error: config/ folder not found!
```

Aplikasi akan melakukan mekanisme load berdasarkan [file-file berikut](#). File-file tersebut berada di dalam satu folder yang akan di-load.

D04 – Save

Jika terjadi perubahan data ketika aplikasi sedang digunakan, tidak baik jika perubahan yang terjadi dituliskan secara manual. Aplikasi harus dapat menulis kembali file-file konfigurasi secara otomatis berdasarkan data aplikasi jika diperlukan. Saat berada di menu utama, gunakan perintah **save <folder>/**. Aplikasi akan membuat **<folder>/** baru jika **<folder>/** tidak ditemukan. Jika **<folder>/** sudah ada, perintah save akan melakukan overwrite terhadap isi folder tersebut.

```
# DISCLAIMER: Tampilan/interface dibebaskan, berikut hanya contoh saja
# KASUS: User melakukan save

>>> save config/

Saving data into config/ folder...
Data saved!

# KASUS: Folder sudah ada

>>> save config/

Saving data into config/ folder...
# Bisa ditambahkan konfirmasi (opsional)
WARNING: config/ folder found, overwrite? (y/n)
Overwriting config/ folder...
Data saved!
```

Pastikan format penulisan data dan ekstensi file sesuai agar dapat di-load kembali. Format dapat dilihat [di sini](#).

Note:

- Manipulasi data program disimpan terlebih dahulu pada memory program. Program hanya melakukan write ke file konfigurasi ketika pengguna menjalankan perintah save.

D05 – Exit

Ketika pengguna keluar dari aplikasi, berikan sebuah opsi save.

```
# DISCLAIMER: Tampilan/interface dibebaskan, berikut hanya contoh saja
# KASUS: Keluar lalu melakukan save

>>> exit

Save data before exiting? (y/n)

# If y

Please input the save folder

>>> config/

# Flow program sama seperti save biasa
Saving data into config/ folder...
Data saved!
Goodbye~

# If n

Goodbye~
```

Struktur Data File

Pada tugas besar ini, terdapat beberapa data file yang akan anda gunakan. Berikut adalah deskripsi dari tiap kolom yang ada pada data file yang diberikan.

File Halaman Web (web_pages.csv)

Nama Atribut	Deskripsi	Tipe Data
id	ID dari halaman web	INTEGER
web_url	URL dari halaman web	STRING
content	Konten dari halaman web	STRING

File Halaman Tertaut (linked_pages.csv)

Nama Atribut	Deskripsi	Tipe Data
id	ID dari halaman web	INTEGER
id_sumber	ID halaman web asal yang memuat tautan (merujuk ke id di web_pages.csv)	INTEGER
id_tujuan	ID halaman web tujuan yang ditautkan (merujuk ke id di web_pages.csv)	INTEGER

Mengingat file .csv menggunakan karakter koma (,) sebagai pemisah kolom dan baris baru sebagai pemisah data, sistem wajib mengimplementasikan ketentuan berikut untuk menjaga integritas data:

1. Atribut *content* pada `web_pages.csv` wajib diapit oleh tanda kutip ganda ("..."). Hal ini bertujuan agar karakter koma yang terdapat di dalam konten tidak disalah artikan sebagai pemisah kolom (delimiter) saat proses parsing.
2. Sistem dilarang menyimpan karakter baris baru (newline) asli di dalam file CSV karena akan dianggap sebagai baris data baru. Gunakan representasi string `\n` di dalam file CSV, kemudian program harus mengubahnya kembali menjadi newline asli saat konten ditampilkan di terminal.

Berikut adalah contoh isi file-csv:

`web_pages.csv`

```
id,web_url,content
1,"mysteryshack.com","SELAMAT DATANG DI MYSTERY SHACK!\nTempat paling ajaib di Nangor Falls (dan satu-satunya yang punya diskon 0%!).\nLihatlah Jackalope asli! Lihatlah batu yang menyerupai wajah Stun!\nPeringatan: Barang yang sudah dibeli tidak dapat dikembalikan, apalagi kalau Anda baru sadar itu cuma barang rongsokan yang ditempel lem."
2,"gideon-tent.org","BACA MASA DEPANMU DI SINI!\n""Lil' Gideon"" tahu segalanya tentangmu.\nBahkan rahasia yang kau sembunyikan di bawah tempat tidur.\nDatanglah ke 'Tent of Telepathy', sekarang juga!"
```

`linked_pages.csv`

```
id,id_sumber,id_tujuan
1,1,2
2,2,1
```

File Konfigurasi (`config.txt`)

```
10 10 5 100
3 1
TAB1 2 1
gideon-tent.org mysteryshack.com
TAB2 1 1
topi-pinus-original.com
TAB3 3 2
batu-ajaib-asli.com topi-pinus-kw.com mysteryshock.com
```

Baris ke-	Deskripsi Konten
1	Konstanta Konfigurasi Global: Berisi nilai batas maksimal untuk sistem, yaitu <code>CACHE_MAX_AMOUNT</code> , <code>TABS_MAX_AMOUNT</code> , <code>DOWNLOAD_MAX_AMOUNT</code> , <code>MAX_WEB_PAGES</code> secara berurutan.
2	Status Tab Global: Menunjukkan jumlah tab yang sedang terbuka dan indeks tab yang sedang aktif saat ini (<i>active tab index</i>). Pada contoh, (3 1) artinya terdapat 3 tab yang aktif (TAB1, TAB2, dan TAB3) dan indeks tab yang sedang dibuka adalah indeks 1 (TAB1)
3	Metadata TAB Terkait: Berisi informasi spesifik mengenai sebuah tab, meliputi: Nama TAB, Total Riwayat (<i>History Count</i>), dan Pointer Posisi Riwayat (<i>Current History Index</i>). Pada contoh, (TAB1 2 1) artinya pada TAB1, terdapat 2 total riwayat, dan TAB1 sedang berada pada indeks ke 1.
4	Data Riwayat URL: Daftar seluruh URL yang tersimpan dalam history stack pada tab tersebut, dipisahkan oleh spasi. Jumlah URL sama dengan <i>History Count</i> pada baris metadata sebelumnya. Pada contoh, (gideon-tent.org mysteryshack.com) artinya riwayat TAB1 memulai dari gideon-tent.org (indeks 1), lalu berpindah ke mysteryshack.com (indeks 2). Karena pada baris ke-3, TAB1 sedang berada pada indeks ke 1, artinya halaman aktif saat ini adalah gideon-tent.org.

Setiap tab direpresentasikan oleh **dua baris data** secara berpasangan: baris pertama untuk **Metadata Tab** dan baris kedua untuk **Daftar Riwayat URL**. Pola ini berulang sebanyak jumlah tab yang aktif (sesuai nilai pada Baris 2).

Note:

- Khusus untuk **STI**, silakan buat penyimpanan data secara **hard coded**, namun tetap menyesuaikan dengan file yang diberi.

Spesifikasi BONUS

Memulihkan ingatan Deeper adalah prioritas utama, namun Mebel merasa browser ini bisa dikembangkan lebih jauh lagi. Terdapat beberapa **Spesifikasi Bonus** yang dapat kalian kerjakan untuk menambah efektivitas pencarian data. Meski begitu, tetaplah fokus pada **Spesifikasi Utama** terlebih dahulu; sebab tanpa pondasi yang kuat, memori Deeper tidak akan pernah kembali. Kerjakan fitur bonus ini hanya jika kalian yakin seluruh sistem utama telah berjalan stabil dan waktu masih berpihak pada kalian.

B01 – Git Best Practice

Penggunaan Git secara spesifikasi wajib, hanyalah sebagai wadah pengumpulan. Satu kali upload file / commit yang berisi seluruh program, lalu membuat Release Tag sudah dianggap cukup. Pada bonus ini, kalian harus menerapkan penggunaan Git secara bersamaan dengan anggota kalian dan menerapkan *best practice* yang ada, yaitu melingkupi Commit Message dan Branching. Video tutorial Git dapat dilihat pada tautan berikut:

 Basic Git Tutorial.mp4

Commit Message

Commit message pada git digunakan sebagai log untuk menjelaskan perubahan yang terjadi pada repository. Commit message yang baik harus jelas, ringkas, dan informatif. Karakteristik commit message yang baik:

Concise: Singkat dan deskriptif

Relevance: Fokus terhadap kenapa perubahan tersebut dilakukan, bukan bagaimana

Clarity: Hindari deskripsi yang ambigu, pesan harus spesifik

Consistency: Pilih dan ikuti format standar (bisa dicari di internet) sehingga mudah untuk dibaca

Commit message yang baik

fix: fix property value display behaviour when editing and selected tool is cursor

 ditramadia committed 2 weeks ago

fix: set width and height rectangle

 bernarduswillson committed 2 weeks ago

Commit message yang jahat

gasssss



Raditss committed 2 years ago

fix



bernarduswillson committed 2 years ago

fix sama gasssss ini maksudnya apaaa????

Contoh standar commit message:

<type>: <description>

type:

feat - Penambahan fitur baru

fix - Bug fix

docs - Perubahan pada dokumentasi (README, laporan, dsb)

style - Formatting, penambahan type safe, dsb

refactor - Refactor kode, ganti nama variable, dsb

test - Penambahan/perubahan script untuk testing

chore - Perubahan pada config file

style: add type safe for RNG algorithm

feat: add RNG algorithm

feat: add database for monsters

refactor: refactor command parser

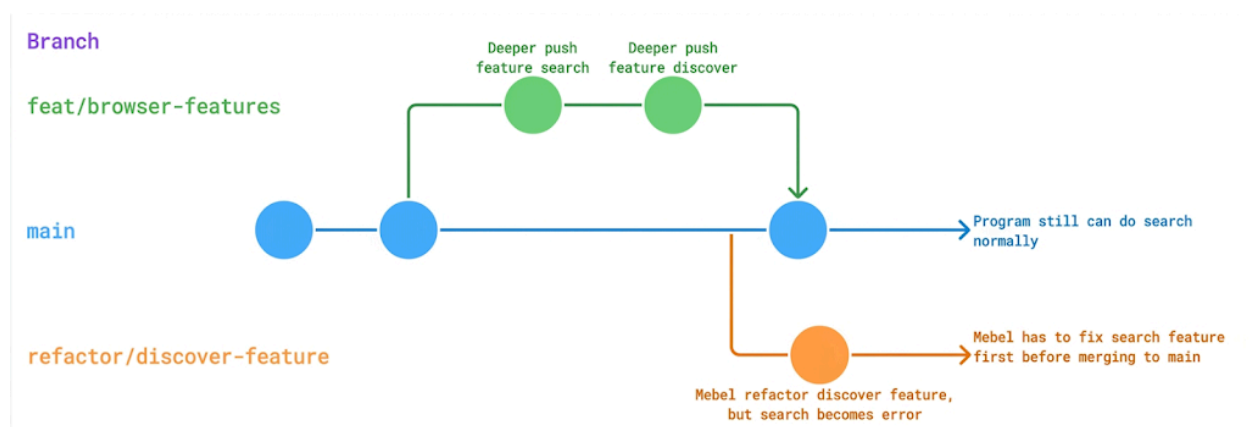
feat: add command parser

Branching

Branching adalah pembagian cabang dari repository kalian. Branching berfungsi untuk memisahkan suatu pekerjaan dengan pekerjaan lainnya (isolasi). Dengan branching, kalian bisa bekerja secara paralel pada branch yang berbeda untuk memitigasi *conflict*.

Misal Deeper dan Mebel tidak menerapkan branching dan hanya bekerja pada branch utama (branch **master** atau **main**). **Deeper** membuat fitur **Search** dan **Discover** lalu melakukan *push*. Kemudian, **Mebel** melakukan *refactor* (perubahan struktur kode) pada fitur **Discover** lalu melakukan *push*. Walaupun fitur **Discover** tetap bekerja, ternyata perubahan tersebut menyebabkan fitur **Search** menjadi *error*. Akhirnya, Deeper tidak bisa menggunakan fitur **Search** dan pekerjaannya terhambat total sebelum Mebel memperbaiki kerusakan tersebut di *branch* utama.

Alternatifnya, Deeper dan Mebel menerapkan *branching*. Deeper telah membuat fitur **Search** dan **Discover** di *branch* terpisah, lalu melakukan *merge* ke *branch* utama setelah dipastikan aman. Saat **Mebel** ingin mengubah fitur **Discover**, ia membuat *branch* baru terlebih dahulu. Ketika ia melakukan *refactor* dan terjadi kesalahan yang merusak fitur **Search**, kode Mebel **tidak berpengaruh** pada kode stabil milik Deeper yang ada di *branch* utama. Mebel dapat memperbaiki fiturnya dengan tenang di *branch* miliknya sendiri, sementara di saat yang bersamaan, Deeper tetap bisa menjalankan fitur **Search** di *branch* utama tanpa gangguan sama sekali.



B02 – Global History

Program menyimpan *Global History* yang mencatat seluruh halaman web yang pernah dibuka oleh pengguna **di semua tab**. *Global History* diimplementasikan menggunakan struktur data **priority queue** dengan prioritas berdasarkan waktu akses terakhir (*most recent access*). Halaman yang paling baru diakses memiliki prioritas tertinggi (berada di posisi teratas *history*). Setiap entri setidaknya menyimpan **URL** beserta **waktu akses terakhirnya**.

Global History bersifat **unik**, setiap URL hanya muncul satu kali di dalam *history*. Jika pengguna membuka kembali halaman yang sudah ada di *history* maka halaman tersebut tidak diduplikasi, melainkan diperbarui prioritasnya (dipindahkan ke posisi teratas sebagai halaman yang paling baru diakses). *Global History* memiliki kapasitas maksimum yang ditentukan oleh variabel **HISTORY_MAX_AMOUNT**. Jika *history* sudah penuh dan terdapat halaman baru yang perlu dicatat, maka entri dengan waktu akses paling lama (prioritas terendah) akan dihapus untuk memberikan ruang bagi entri baru.

History tercatat setiap kali pengguna berhasil membuka suatu halaman, baik melalui **open <url>**, **openlinked <x>**, maupun navigasi **back/forward** (juga **back <x>/forward <x>**). Saat tab ditutup (**closetab**), halaman yang pernah dibuka di tab tersebut tetap tercatat di *Global History*. Pengguna dapat melihat *history* dengan perintah **history** dari menu utama (*home*).

Berikut merupakan contoh kasus-kasus yang mungkin terjadi untuk **Global History**.

```
# DISCLAIMER: Tampilan/interface dibebaskan, berikut hanya contoh saja
```

```
# KASUS 1: User belum pernah membuka halaman apapun.
```

```
>>> open mysteryshack.com
```

```
Riwayat kosong, belum ada halaman yang pernah dibuka.
```

```
# KASUS 2: User membuka beberapa halaman, lalu melihat history.
```

```
# Perintah 1: User membuka halaman web mysteryshack.com
```

```
>>> open mysteryshack.com
```

```
SELAMAT DATANG DI MYSTERY SHACK!
```

```
...
```

```
Linked pages:
```

```
[1] batu-ajaib-asli.com
```

```
[2] topi-pinus-original.com
```

```

# Perintah 2: User membuka linked page dari halaman yang sedang dibuka
>>> openlinked 1

BATU AJAIB ASLI - LANGSUNG DARI GUA MISTERIUS!
...

# Perintah 3: User kembali ke menu utama
>>> home

# Perintah 4: User membuka halaman web six.itb.ac.id
>>> open six.itb.ac.id

Halaman SIX ITB ...

# Perintah 5: User kembali ke menu utama
>>> home

# Perintah 6: User melihat history
>>> history

Riwayat halaman yang pernah dikunjungi:
[1] six.itb.ac.id (14:03)
[2] batu-ajaib-asli.com (14:02)
[3] mysteryshack.com (14:01)

```

```

# KASUS 3: User membuka halaman yang sudah ada di history
# (prioritas dan waktu akses diperbarui, tidak diduplikasi).
# Perintah 1: User melihat history
>>> history

Riwayat halaman yang pernah dikunjungi:
[1] six.itb.ac.id (14:03)
[2] batu-ajaib-asli.com (14:02)
[3] mysteryshack.com (14:01)

# Perintah 2: User membuka kembali halaman yang sudah pernah dikunjungi
>>> open mysteryshack.com

SELAMAT DATANG DI MYSTERY SHACK!
...
Linked pages:
[1] batu-ajaib-asli.com
[2] topi-pinus-original.com

# Perintah 3: User kembali ke menu utama
>>> home

# Perintah 4: User melihat history
>>> history

Riwayat halaman yang pernah dikunjungi:
[1] mysteryshack.com (14:05)
[2] six.itb.ac.id (14:03)
[3] batu-ajaib-asli.com (14:02)

# Penjelasan: mysteryshack.com naik ke posisi teratas dengan waktu akses terbaru.
# Entri tidak diduplikasi, hanya prioritas dan waktunya yang diperbarui.

```

```

# KASUS 4: History mencatat akses dari semua tab (global).
# Perintah 1: User membuka halaman di TAB1

```

```

>>> open mysteryshack.com

...

# Perintah 2: User kembali ke menu utama
>>> home

# Perintah 3: User membuat tab baru
>>> newtab

Tab baru (TAB2) berhasil dibuat!

# Perintah 4: User berpindah ke TAB2
>>> switchtab TAB2

Tab saat ini berhasil diganti ke TAB2.

# Perintah 5: User membuka halaman di TAB2
>>> open six.itb.ac.id

...

# Perintah 6: User kembali ke menu utama
>>> home

# Perintah 7: User melihat history
>>> switchtab TAB2

Riwayat halaman yang pernah dikunjungi:
[1] six.itb.ac.id (14:10)
[2] mysteryshack.com (14:08)

# Penjelasan: Walaupun dibuka di tab berbeda, keduanya masuk ke satu Global History yang sama.

```

```

# KASUS 5: Navigasi back/forward juga memperbarui history.
# Kondisi awal: Di suatu tab, user sudah membuka mysteryshack.com
# lalu batu-ajaib-asli.com secara berurutan. History saat ini:
#     [1] batu-ajaib-asli.com (14:12)
#     [2] mysteryshack.com (14:11)

# Perintah 1: User melakukan back ke halaman sebelumnya
>>> back

BACK: KEMBALI KE HALAMAN mysteryshack.com
SELAMAT DATANG DI MYSTERY SHACK!
...

# Perintah 2: User kembali ke menu utama
>>> home

# Perintah 3: User melihat history
>>> history

Riwayat halaman yang pernah dikunjungi:
[1] mysteryshack.com (14:14)
[2] batu-ajaib-asli.com (14:12)

```

```
# Penjelasan: Meskipun dibuka via perintah back, history tetap diperbarui
# sehingga mysteryshack.com naik ke posisi teratas dengan waktu akses terbaru.
```

```
# KASUS 6: History sudah penuh (jumlah entri = HISTORY_MAX_AMOUNT),
#         halaman baru yang dibuka akan menghapus entri paling lama.
# Kondisi awal: HISTORY_MAX_AMOUNT = 3, history saat ini:
#     [1] six.itb.ac.id (14:20)
#     [2] batu-ajaib-asli.com (14:15)
#     [3] mysteryshack.com (14:10)
```

```
# Perintah 1: User membuka halaman baru yang belum ada di history
>>> open topi-pinus-original.com
```

```
TOPI PINUS IKONIK ORIGINAL!
...
```

```
# Perintah 2: User kembali ke menu utama
>>> home
```

```
# Perintah 3: User melihat history
>>> history
```

```
Riwayat halaman yang pernah dikunjungi:
[1] topi-pinus-original.com (14:22)
[2] six.itb.ac.id (14:20)
[3] batu-ajaib-asli.com (14:15)
```

```
# Penjelasan: mysteryshack.com (entri paling lama) dihapus karena history
# sudah penuh, memberikan ruang bagi topi-pinus-original.com.
```

Jika mengimplementasikan bonus ini, silahkan sesuaikan penyimpanan data pada config.txt, format dibebaskan.

B03 – Advanced Search

Pada implementasi umumnya di dunia nyata, *search engine* (di internet) tidak hanya terbatas pada pencarian URL web, tetapi juga mencakup pencarian konten dari web tersebut. Hal ini bertujuan agar pengguna dapat menemukan web yang dimaksud dengan lebih mudah.

Untuk bonus ini, kembangkan fungsionalitas [Search](#) agar pencarian juga berlaku pada **konten web**. Ubah juga fungsi pencarian agar bersifat **case-insensitive** dan berlaku untuk **substring** (Pencocokkan string query dilakukan pada keseluruhan

string target: URL dan konten web, tidak hanya dimulai dari awal string target. Ini disebut juga dengan **string-matching**).

```
# DISCLAIMER: Tampilan/interface dibebaskan, berikut hanya contoh saja

# KASUS 1: User mencari halaman web dengan query "Shack"
# Web mysteryshack.com ditemukan karena URL mengandung string "shack".

>>> search Shack

Search result(s) for "Shack":
- mysteryshack.com

# KASUS 2: User mencari halaman web dengan query "batu".
# Ada dua web yang ditemukan:
#
# - Web mysteryshack.com ditemukan karena konten web tersebut
#   mengandung string "batu" pada bagian:
#     ...
#     Lihatlah Jackalope asli! Lihatlah batu yang menyerupai wajah Stun!
#     ...
#
# - Web batu-ajaib-asli.com ditemukan karena URL mengandung
#   string "batu",

>>> search batu

Search result(s) for "batu":
- mysteryshack.com
- batu-ajaib-asli.com
```

B04 – Bookmark Manager

Pengguna seringkali menemukan halaman web yang sangat penting dan ingin menyimpannya agar dapat diakses kembali dengan cepat di lain waktu tanpa harus mencari melalui *search engine* atau riwayat. Perintah yang dapat digunakan adalah sebagai berikut:

1. **add_bookmark <alias> <url>**: Menambahkan URL tertentu ke dalam daftar bookmark dengan nama alias. Jika URL tidak ditemukan di database atau alias sudah digunakan, tampilkan pesan kesalahan.
2. **view_bookmark**: Menampilkan seluruh daftar bookmark yang tersimpan dalam format tabel atau list (menampilkan alias dan URL).

3. **delete_bookmark <alias>**: Menghapus entri bookmark berdasarkan nama alias yang diberikan.
4. **open_bookmark <alias>**: Perintah cepat untuk langsung membuka halaman web yang sudah tersimpan di bookmark berdasarkan aliasnya.

Jika mengimplementasikan bonus ini, silahkan sesuaikan penyimpanan data pada config.txt, format dibebaskan.

BXX – Kreativitas

Crai, agen dari Cipher, sangat menyukai program yang tidak hanya berfungsi dengan baik, tetapi juga memiliki "jiwa" dan keunikan tersendiri. Sebagai pengembang browser misterius ini, kalian diberikan kebebasan penuh untuk berkreasi di luar fungsionalitas yang telah ditetapkan.

Silakan ajukan fitur kreatif kalian kepada Asisten pembimbing atau melalui Sheets QNA untuk memastikan fitur tersebut layak mendapatkan poin bonus. Perlu diingat bahwa fitur tambahan ini tidak boleh merusak atau mengubah alur fungsionalitas utama. Jika fitur yang kalian ajukan cukup mengesankan dan dieksekusi dengan rapi, Cipher mungkin akan memberikan "imbalan" nilai yang setimpal.

Berikut adalah beberapa contoh ide kreativitas yang dapat diimplementasikan:

- Penggunaan ASCII Art, teks berwarna (ANSI escape codes), atau antarmuka menu yang lebih interaktif.
- Implementasi sistem view_count untuk setiap halaman web. Data kunjungan harus bersifat persisten (tersimpan di database) dan dapat digunakan untuk fitur tambahan seperti menampilkan daftar halaman terpopuler (Trending).
- [Khusus IF, EL, EB] Sistem Incognito Mode (riwayat tidak dicatat) atau melakukan enkripsi sederhana pada file konfigurasi/save data menggunakan algoritma kriptografi buatan sendiri.
- [Khusus STI] Fitur crawl <depth> yang otomatis membuka semua linked pages dari satu URL sampai kedalaman tertentu dan menampilkannya dalam bentuk daftar atau pohon.
- Dan lain-lain...

Ketentuan dan Batasan

A. Ketentuan Umum

Folder terpusat:  Tugas Besar IF1210

1. Tugas dikerjakan berkelompok dengan anggota kelompok sebanyak 5–6 orang per kelompok dengan anggota kelompok yang telah ditentukan oleh asisten. Link **pembagian kelompok** serta **pemilihan asisten** dapat dilihat pada tautan berikut:

 Pembagian Kelompok dan Asisten IF1210 2026 – Tugas Besar 1

2. **Template MoM Asistensi** dan **Laporan Tugas Besar** dapat dilihat pada tautan berikut:

 IF1210 Template Laporan.docx

 IF1210 Form Asistensi

3. Pertanyaan terkait Tugas Besar, dapat diajukan dalam **Spreadsheets QNA** yang ada pada tautan berikut:

 QnA IF1210 2026 – Tugas Besar 1

Mekanisme pengerjaan silakan perhatikan bagian [Pengumpulan dan Deliverables](#)

B. Batasan Pengerjaan

1. Menggunakan bahasa **C**
2. Hanya boleh melakukan include library dasar C, seperti **stdio.h**, **stdlib.h**, dan **string.h**, serta modul-modul serta fungsi yang telah dibuat sendiri.
3. Tidak boleh menggunakan library eksternal dalam bentuk apapun. (Apabila ada library lain yang dibutuhkan, bisa dikonfirmasi terlebih dahulu ke asisten melalui sheets QnA)
4. Hanya boleh memakai fungsi-fungsi bawaan dari C yang diajarkan di kelas. Silakan mengimplementasikan sendiri fungsi tambahan yang dibutuhkan.
5. **Tidak boleh membuat temporary** (baik berupa file .txt, .csv, .dat, atau ekstensi file lainnya) **sebagai tempat penyimpanan sementara** selama

program berjalan. Seluruh penyimpanan data sementara harus dilakukan menggunakan struktur data yang dibuat sendiri di dalam memori (RAM).

Penggunaan file eksternal hanya diperbolehkan pada saat fungsi load dan save saja.

6. Program berjalan sebagai satu kesatuan. Main program hanya boleh satu. Main program ini akan memanggil fungsi/prosedur dari modul lain yang berada di file lain.
7. Tugas besar harus diimplementasikan dalam **paradigma prosedural dan fungsional** seperti yang diajarkan di kelas. Tidak boleh menggunakan paradigma lain seperti OOP.
8. Environment pengerjaan dibebaskan. Akan tetapi program harus dapat dikompilasi di **Linux / Unix** (untuk pengguna windows, program harus dapat dikompilasi di [WSL](#)). Kompilasi akan dilakukan dengan menggunakan **makefile**. (Jika tidak bisa di-compile di Linux/Unix system, asisten tidak dapat menilai 🙏)

C. Timeline Pengerjaan

Kegiatan	Deadline	
	Tanggal	Jam (WIB)
Release Tugas Besar	27 April 2026	12.10
Pengisian Asisten Kelompok	29 April 2026	23.59
Pengerjaan Tugas Besar		
- Deadline Asistensi 1 (Klarifikasi Spek Tugas Besar)	6 Mei 2026	23.59
- Deadline Asistensi 2	18 Mei 2026	23.59
- Pengumpulan <i>deliverables</i> (softcopy laporan dan source code program)	25 Mei 2026	12.10
Penilaian Tugas Besar		
- Demo Tugas Besar	TBA	-

D. Asistensi

1. Setiap kelompok akan dibantu oleh seorang asisten pembimbing. Asistensi wajib dilakukan kepada asisten pembimbing.
2. Anggota kelompok menghubungi asisten pembimbing untuk mengajukan asistensi atau bertanya.
3. Asisten pembimbing bertugas untuk membimbing pengerjaan Tugas Besar dan melakukan klarifikasi terhadap spesifikasi Tugas Besar.
4. Asisten penguji demo bertugas untuk memberikan penilaian saat demo Tugas Besar.

5. Kelompok Tugas Besar wajib melakukan Asistensi 1 dan 2 sebagai berikut.
 - a. Asistensi 1 (untuk klarifikasi spesifikasi tugas): paling lambat **6 Mei 2026**. Wajib.
 - b. Asistensi 2: paling lambat hari **18 Mei 2026**. Wajib.
 - c. Asistensi 3: Opsional, silakan hubungi asisten pembimbing untuk mengajukan asistensi tambahan jika diperlukan.
6. Untuk semua asistensi, maksimal menghubungi asisten pembimbing J-24 sebelum waktu asistensi yang diajukan.
7. Setiap kali asistensi, peserta maupun asisten diharuskan mengisi form asistensi dengan lengkap. Form asistensi harus diisi dan diparaf saat asistensi berlangsung atau segera setelah asistensi berakhir. Form Asistensi dapat diunduh di situs folder [Tugas Besar IF1210](#).

E. Demo

1. Setelah masa deadline pengerjaan Tugas Besar, akan dilaksanakan Demo Tugas Besar secara sinkron dengan asisten masing-masing.
2. Demo bertujuan untuk mempresentasikan hasil pengerjaan tugas besar.
3. Program akan dijalankan di depan asisten dan akan diuji fungsionalitas, validasi, dan juga pemahaman kalian terhadap program.
4. Mekanisme demo akan dijelaskan lebih lanjut di kemudian hari.

Pengumpulan dan Deliverables

1. Untuk tugas ini Anda diwajibkan menggunakan git version control system dengan menggunakan sebuah repository private di Github Classroom "Labpro-23". Silakan akses Github Classroom pada:

[Github Classroom - Labpro 23](#)

Notes:

Dalam sehari, hanya bisa kurang lebih 50 orang yang bergabung ke github classroom. Jadi, gantian ya masuknya 🙏

2. Silakan membuat Group dengan format nama "KXX-Y", dengan penjelasan:
XX: Nomor kelas (ditulis dalam 2 digit, misal: 03)
Y: Kode kelompok (ditulis dalam 1 huruf, misal: A)
Contoh: K03-A
3. Pembuatan Group dilakukan **satu kali** untuk tiap kelompok, setelah dibuat, anggota kelompok lain **tidak perlu membuat Group lagi** dan dapat langsung bergabung dengan Group yang sudah dibuat.
4. Akan terdapat repository private yang dibuat secara **otomatis** untuk tiap Group, silakan gunakan repository tersebut untuk pengerjaan Tugas Besar. Repository tersebut akan memiliki nama: **if1210-tubes-2026-kxx-y**
5. Repository berisi:
 - Folder **src**: Berisi kode program / fungsi-fungsi.
 - Folder **config**: Contoh data yang dapat di load.
 - Folder **doc**: Berisi laporan dalam format .pdf.
 - Folder **bin**: Sebagai folder tempat executable hasil kompilasi. Executable tidak perlu di-push ke dalam repository.
 - **Makefile**: file konfigurasi yang berisi sekumpulan aturan untuk mengotomatisasi proses kompilasi, build, menjalankan, dan membersihkan program.

- **README.md:** Berisi setidaknya penjelasan ringkas program, cara kompilasi program, cara menjalankan program, pembagian tugas, dan daftar fitur beserta status pengerjaannya (selesai / tidak).

6. Pengumpulan dilakukan dengan membuat **release** dengan major version 1 seperti **v1.x**. Versi patch dapat digunakan jika ada perbaikan. Contoh: Jika ada revisi pada versi **v1.0**, selama tidak melewati deadline, dipersilahkan melakukan revisi kode dan melakukan release lagi dengan versi **v1.1**. Penilaian akan dilakukan terhadap versi rilis terakhir.

Untuk penyusunan Laporan Tugas Besar, disediakan template laporan pada folder [Tugas Besar IF1210](#). Laporan dikumpulkan dalam format pdf. Form Asistensi yang telah ditandatangani asisten dijadikan satu dengan sebuah aplikasi PDF Combiner dengan laporan sebagai lampiran. Progress pengerjaan (milestone) juga **wajib** dilampirkan pada laporan. Progress minimal yang harus dilaporkan kepada asisten pembimbing adalah sebagai berikut:

Tanggal			DD-MM-YYYY
No	Fitur	Progress (0-100%)	Keterangan (Komponen fitur yang telah selesai ataupun kendala)
1.			
2.			
3.			
4.			
6.			
7.			
8.			
9.			
10.			

11.			
-----	--	--	--

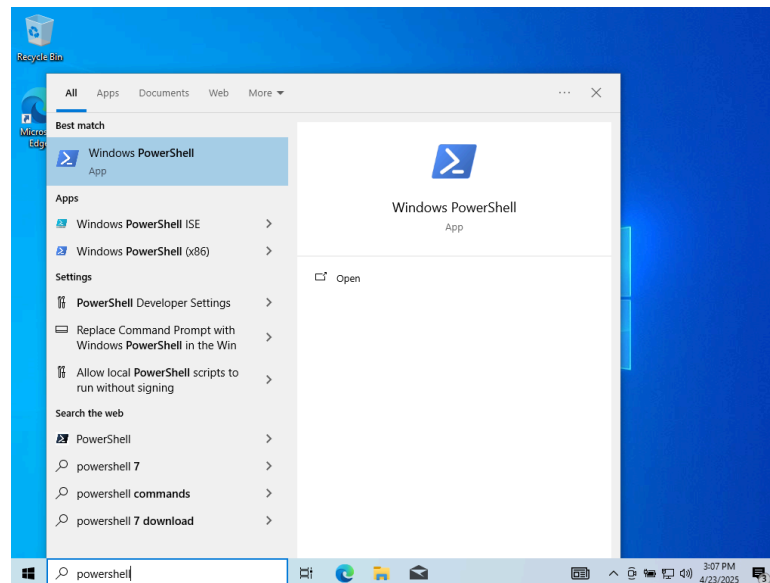
Ketentuan pengumpulan dokumen adalah sebagai berikut:

- a. **Milestone 1:** Dokumen draft laporan sementara dengan format nama
IF1210_LaporanTB_MS1_K[XX-Y].pdf
- b. **Milestone 2:** Dokumen laporan final dengan format nama
IF1210_LaporanTB_MS2_K[XX-Y].pdf

Panduan Instalasi WSL + Makefile

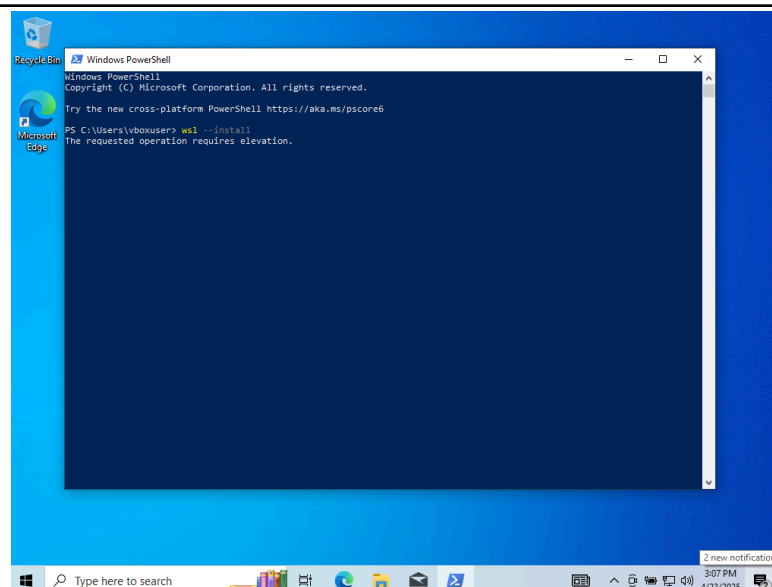
A. Windows 10/11

1. Buka terminal Powershell

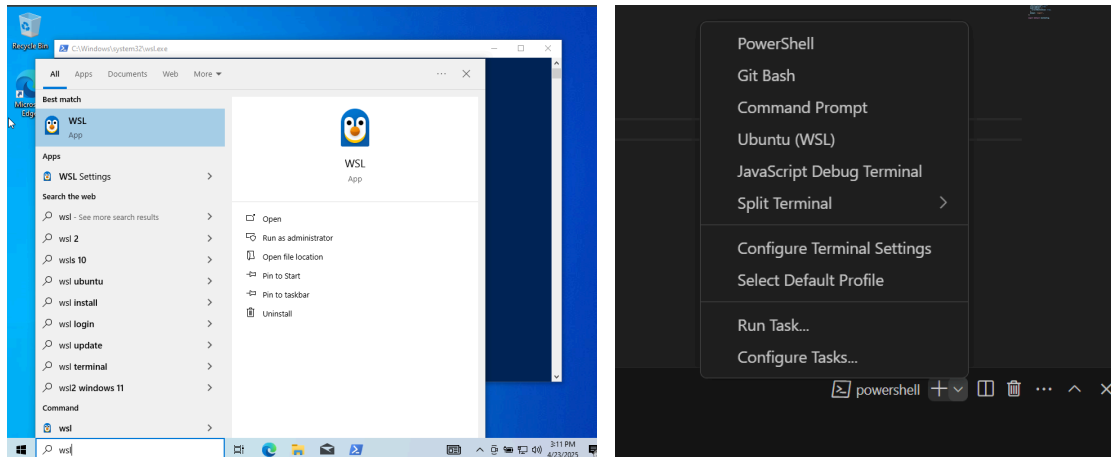


2. run perintah berikut ini untuk menginstall wsl

```
wsl --install
```



3. Buka terminal WSL yang sudah diinstal barusan (Jika menggunakan vscode juga akan ada).



4. Update terlebih dahulu list package yang tersedia

```
sudo apt update
```

5. Install package essential (sudah termasuk makefile dan c)

```
sudo apt install build-essential
```

6. Makefile dan compiler gcc sudah terinstall. Coba dengan run perintah berikut

```
make --version  
gcc --version
```

7. Referensi tambahan: [WSL Installation](#) (Recommended to upgrade your WSL 1 to WSL 2)

B. Mac

1. Tidak perlu ada setup tambahan
2. Pastikan `make` dan `gcc` sudah terinstall dengan menggunakan command berikut

```
make --version  
gcc --version
```

Referensi

- [Wikipedia - Linear Congruential Generation \(LCG\)](#)
- [Case-Sensitive and Case-Insensitive String](#)
- [GeeksforGeeks - Arguments in C](#)
- [GeeksforGeeks - C Programming Concepts Used as Data Structures](#)
- [Cache Replacement Algorithms: How To Efficiently Manage The Cache Storage](#)
- [Git Best Practices](#)
- [Git Semantic Commit Message](#)
- [Makefile - Reference 1](#)
- [Makefile - Reference 2](#)
- [Makefile + GCC - Reference 3 \(recommended after having basic makefile understanding\)](#)
- [Microsoft - WSL Installation](#)
- [Regular Expression](#)